Advanced Terraform on AWS Lab 8
Landing Zone Consumption

Expected Duration

Phase 1 of this lab is expected to take between #-# minutes
Phase 2 of this lab is expected to take #-# minutes
Phase 3 of this lab is expected to take #-# minutes
Phase 4 of this lab is expected to take #-# minutes
Phase 5 of this lab is expected to take #-# minutes

Total expected lab time is therefore #-# minutes

Try to avoid skipping straight to actionable lab steps as the logic of what you are trying to achieve may be lost and the lab learning value will therefore be reduced.

Prerequisites

This lab assumes the existence of a remote state backend, as created in Lab 4. Run Phase 2 of Lab 4 now if you do not have remote state backend configured.

This lab also assumes that labs 6a, 6b and 7 have been completed successfully.

Teaching Points

This lab focuses on how workload teams correctly adopt and reconcile with a centrally governed AWS Landing Zone. It continues the operating model established in the previous labs: platform resources are centrally owned, workload teams consume them, and Terraform is used to make ownership, impact and risk visible.

Key concepts reinforced in this lab:

- Terraform adoption often begins with existing workloads. Workload teams may inherit infrastructure that was created before platform standardisation. This lab reflects that reality by starting with unmanaged legacy resources.
- Terraform state defines workload ownership. Workload teams are responsible only for resources present in their Terraform state. Import is the mechanism by which workload ownership is recovered without rebuilding infrastructure.
- Generated configuration supports import, not design authority. Terraform-generated configuration represents observed AWS state, not architectural intent. It must be reviewed and aligned with Terraform resource models and platform constraints.
- Landing Zones are platform-owned and immutable. Workload teams consume Landing Zone resources using data sources. They must not import, modify or assume ownership of platform infrastructure.
- Workload migration follows a replace-then-retire model. Workload teams create Landing Zone-aligned replacements first, then explicitly retire non-compliant legacy resources they own.
- Terraform enables governance by surfacing impact. Terraform plans make destructive workload changes explicit, enabling review, approval and risk assessment. Terraform does not conceal or automatically fix architectural decisions.

By the end of this lab, you should be able to clearly distinguish:

- Platform-owned resources: the Landing Zone VPC, shared subnets and shared routing model
- Workload-owned resources: imported legacy assets and new workload deployments
- Terraform's role as a reconciliation and governance-enabling tool within a platform operating model

Phase 1 - Manual Deployment

In this phase you simulate what often exists before an organisation adopts a centralised Terraform-driven operating model. A Business Unit has deployed a workload manually using bespoke networking and security, with no Terraform ownership.

The folder **lab8\guided\seed** contains Terraform code used purely to seed AWS with resources. Once the resources are created, the Terraform state file produced in this phase will be deliberately destroyed. From that point onward, the resources should be treated as manually created infrastructure.

The configuration deploys:

- A legacy VPC
- A legacy subnet outside the Landing Zone
- A legacy security group
- A legacy elastic network interface

All resources are named with a legacy- prefix so they can be easily identified later.

Initialise Terraform and deploy the resources

1. From **lab8\guided\seed**, review **terraform.tfvars** and confirm the AWS region is set to us-west-2, unless your lab environment states otherwise.
2. Run:

   ```
   cd c:\qatfadvaws\lab8\guided\seed
   terraform init
   terraform apply --auto-approve
   ```

3. In the AWS Console, confirm that the following resources exist:
   - A VPC named legacy-vpc
   - A subnet named legacy-subnet
   - A security group named legacy-sg
   - An elastic network interface named legacy-eni

Deliberately remove Terraform state

To simulate a real-world unmanaged environment, you will now remove Terraform's memory of the deployment.

4. Delete the entire **lab8\guided\seed** folder.

This concludes the seeding phase. In the next phase, the Business Unit will establish a remote Terraform backend in preparation for importing these unmanaged resources into Terraform state.

Phase 2 - Create and Consume a Remote State Backend

In this phase, the Business Unit adopts Terraform by first creating a team-safe remote backend for state storage. At this point, no workload resources are being managed yet. The goal is to prepare Terraform to safely manage infrastructure in the next phases.

Create a Business Unit remote backend

5. Navigate to artifacts\bu-state-backend-bootstrap. This folder contains remote backend code. It is a standalone bootstrap configuration, similar to the one used in Lab 4.
6. Open **terraform.tfvars** and update the following values:
   - bucket_suffix = **"change-me"**
7. From the bootstrap folder, run:

   ```
   cd c:\qatfadvaws\artifacts\bu-state-backend-bootstrap
   terraform init
   terraform apply --auto-approve
   ```

   This creates an S3 bucket for Business Unit state storage. The lab uses native S3 state locking with use_lockfile=true, so no DynamoDB lock table is required.

Configure the Business Unit Terraform backend

8. Navigate to lab8\guided and review **providers.tf**
9. Update the backend configuration with your bucket name and region. The bucket name should match the bucket created by the bootstrap configuration.

   ```
   terraform {
    backend "s3" {
     bucket       = "bu-tfstate-<your-suffix>"
     key          = "business-unit.tfstate"
     region       = "us-west-2"
     use_lockfile = true
    }
   }
   ```

10. Initialise Terraform using the remote backend:

    ```
    cd c:\qatfadvaws\lab8\guided
    terraform init
    ```

11. You should now be able to confirm:

    - A remote backend exists in S3
    - Terraform initialises successfully using that backend
    - State will be stored centrally in business-unit.tfstate
    - No workload resources are managed yet

    The Business Unit is now ready to import existing AWS resources into Terraform.

Phase 3 - Import Manually Created Resources

In this phase you import the legacy resources that already exist in AWS into the Business Unit's remote Terraform state. These resources were manually created from Terraform's perspective because the local state was deliberately removed in Phase 1.

What you are about to do

- What: Import existing AWS resources into Terraform state using AWS resource IDs.
- Why: Terraform can only manage what it has in state. Import is how workload ownership is recovered without rebuilding.
- Success looks like: terraform state list shows the legacy resources in the BU remote backend.

Collect the AWS resource IDs

12. Record the ids of the legacy resources created in Phase 1:

legacy-vpc:

```
aws ec2 describe-vpcs `
 --region us-west-2 `
 --filters "Name=tag:Name,Values=legacy-vpc" `
 --query "Vpcs[0].VpcId" `
 --output text
```

legacy-subnet:

```
aws ec2 describe-subnets `
 --region us-west-2 `
 --filters "Name=tag:Name,Values=legacy-subnet" `
 --query "Subnets[0].SubnetId" `
--output text
```

legacy-sg:

```
aws ec2 describe-security-groups `
 --region us-west-2 `
 --filters "Name=tag:Name,Values=legacy-sg" `
```

```
    --query "SecurityGroups[0].GroupId" `
    --output text

legacy-eni:
    aws ec2 describe-network-interfaces `
    --region us-west-2 `
    --filters "Name=tag:Name,Values=legacy-eni" `
    --query "NetworkInterfaces[0].NetworkInterfaceId" `
    --output text
```

Create import blocks

13. In lab8\guided, create a new file named imports.tf.
14. Paste in the following, replacing the placeholders with your copied IDs:

```
import {
  to = aws_vpc.legacy
  id = "<LEGACY_VPC_ID>"
}

import {
  to = aws_subnet.legacy
  id = "<LEGACY_SUBNET_ID>"
}

import {
  to = aws_security_group.legacy
  id = "<LEGACY_SECURITY_GROUP_ID>"
}

import {
  to = aws_network_interface.legacy
  id = "<LEGACY_ENI_ID>"
}
```

Generate starter configuration from the import plan

In this step, Terraform will read the import intent, inspect the real AWS resources and generate best-effort resource blocks into a file.

15. At lab8\guided, run:

```
cd c:\qatfadvaws\lab8\guided
terraform init
terraform plan --generate-config-out=raw-legacy-generated.tf
```

Expected outcome:

- Terraform produces a plan that includes import actions
- Terraform writes **raw-legacy-generated.tf**
- The generated file may be verbose and may require review

16. A reviewed and cleaned version of this raw file has been provided for you, **cleaned-legacy-generated.txt**. Compare **raw-legacy-generated.tf** and **cleaned-legacy-generated.txt.**

Understanding the Cleaned Generated Configuration

Terraform's generated configuration is a starting point, not a final design. It reflects what Terraform observed in AWS, including many values that are either computed by AWS, unnecessary to manage explicitly, or too brittle to keep in configuration.

The cleaned file keeps the important architectural intent but removes noise and replaces fixed AWS identifiers with Terraform references.

Change 1 – Remove provider-generated noise

The generated file included many values such as:

```
tags_all
region
null attributes
empty lists
computed settings
```

These values were observed from AWS, but they do not need to be managed directly. By removing these generated values, the configuration becomes shorter, clearer and easier to maintain.

Change 2 – Replace hard-coded AWS IDs

The generated configuration used hard-coded IDs such as:

```
vpc_id = "vpc-024d81e21a0ac6a04"
subnet_id = "subnet-0a45b769ea490db74"
security_groups = ["sg-0be8a16029efea389"]
```

These IDs are real AWS identifiers but keeping them in Terraform makes the configuration brittle. Instead, the cleaned configuration uses Terraform references:

```
vpc_id = aws_vpc.legacy.id
subnet_id = aws_subnet.legacy.id
security_groups = [aws_security_group.legacy.id]
```

This allows Terraform to understand the relationship between the resources.

Change 3 – Remove fixed private IP values

The generated network interface included values such as:

```
private_ip      = "10.50.1.39"
private_ips     = ["10.50.1.39"]
private_ip_list = ["10.50.1.39"]
```

These values describe the current address AWS assigned to the ENI. For this lab, the specific private IP address is not important. What matters is that the ENI exists in the correct subnet and uses the correct security group.

Removing the fixed private IP values allows AWS to continue managing the address allocation unless the workload specifically requires a static private IP.

Change 4 – Keep only meaningful design settings

The cleaned configuration keeps values that describe the intended structure:

```
cidr_block
availability_zone
name
description
ingress rules
egress rules
tags
```

These values are meaningful because they describe the legacy design being adopted into Terraform. The objective is not to reproduce every observed attribute. The objective is to create maintainable Terraform that accurately represents the resources the Business Unit now owns.

17. Delete file **raw-legacy-generated.tf**
18. Rename **cleaned-legacy-generated.txt** to **cleaned-legacy-generated.tf**
19. Run terraform plan again. No errors should be reported.

Key takeaway: configuration generated during import is not production-ready. It reflects what Terraform can observe, not what Terraform should configure. Human review is required.

Validate state before import

20. Before performing the import, confirm that the Business Unit remote backend currently contains no managed resources:

terraform state list

Expected output: no state file exists.

At this point AWS contains legacy resources but Terraform does not yet own them. Import intent has been declared, but state has not been created/modified.

Apply the import

21. Run **terraform apply**

Terraform should display a plan similar to:

```
Plan: 4 to import, 0 to add, 0 to change, 0 to destroy.
```

22. When prompted, type **yes**.

    Terraform will now read the real AWS resources, write them into the remote backend state and associate them with the configuration you have validated.

Validate import completion

23. After apply completes, run **terraform state list**

    Expected output should be:

    - aws_vpc.legacy
    - aws_subnet.legacy
    - aws_security_group.legacy
    - aws_network_interface.legacy

    This confirms that legacy resources are now tracked in Terraform state and ownership has been re-established without rebuilding infrastructure.

Confirm no-op alignment

24. Run **terraform plan**

    Expected result: No changes. Infrastructure is up-to-date.

    This proves that configuration, remote state and AWS reality are aligned.

File clean-up

25. Following the successful import, clean the working folder:

- Delete imports.tf
- Rename cleaned-generated.tf to main.tf

26. Run **terraform fmt**

Phase outcome: the Business Unit now owns the imported legacy resources in a remote backend. In the next phase, the Business Unit will deploy new workloads directly into the Landing Zone.

Phase 4 - Consume the Landing Zone

In this phase you deploy a new Business Unit workload into the DEV Landing Zone. You will not recreate, modify, or import any Landing Zone resources. The Landing Zone is treated as authoritative and immutable.

Confirm the target Landing Zone environment

The Landing Zone exists across DEV, TEST and PROD. For this lab you must consume only the DEV Landing Zone.

You will reference DEV Landing Zone resources using data sources. In all steps below, use DEV Landing Zone values only.

Review the starting files

27. Navigate to c:\qatfadvaws\lab8\guided and review:

- providers.tf - provider and remote backend
- variables.tf - intentionally empty
- terraform.tfvars - intentionally empty
- main.tf - populated with the imported legacy resources

Update terraform.tfvars with DEV Landing Zone values

28. Open terraform.tfvars and set the Landing Zone lookup values:

```
region = "us-west-2"

# Landing Zone (dev)
lz_vpc_name    = "landing-zone-dev-vpc"
lz_subnet_name = "landing-zone-dev-subnet-app"

# Business Unit workload
workload_name  = "bu-workload-dev"
```

Update variables.tf

29. Open variables.tf and add:

```
variable "region" {
  description = "AWS region used for the lab."
  type        = string
}

variable "lz_vpc_name" {
  description = "Name tag of the DEV Landing Zone VPC."
  type        = string
}

variable "lz_subnet_name" {
  description = "Name tag of the DEV Landing Zone application subnet."
  type        = string
}
```

```
variable "workload_name" {
  description = "Name prefix for the Business Unit workload."
  type        = string
}
```

Add Landing Zone data sources and workload resources

30. Open **main.tf** and add the following configuration beneath the imported legacy resources.

Look up the DEV Landing Zone VPC and subnet:

```
data "aws_vpc" "lz" {
 filter {
  name   = "tag:Name"
  values = [var.lz_vpc_name]
 }
}

data "aws_subnet" "lz_app" {
 filter {
  name   = "tag:Name"
  values = [var.lz_subnet_name]
 }

 filter {
  name   = "vpc-id"
  values = [data.aws_vpc.lz.id]
 }
}
```

Create workload-owned security and compute resources inside the Landing Zone:

```
data "aws_ami" "al2023" {
 most_recent = true
 owners      = ["amazon"]

 filter {
  name   = "name"
  values = ["al2023-ami-2023.*-x86_64"]
 }

 filter {
  name   = "architecture"
  values = ["x86_64"]
 }
}

resource "aws_security_group" "bu_workload" {
```

```
  name        = "${var.workload_name}-sg"
  description = "Business Unit workload security group"
  vpc_id      = data.aws_vpc.lz.id

  egress {
   from_port   = 0
   to_port     = 0
   protocol    = "-1"
   cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
   Name  = "${var.workload_name}-sg"
   Owner = "business-unit"
  }
}

resource "aws_network_interface" "bu_workload" {
  subnet_id       = data.aws_subnet.lz_app.id
  security_groups = [aws_security_group.bu_workload.id]

  tags = {
   Name  = "${var.workload_name}-eni"
   Owner = "business-unit"
  }
}

resource "aws_instance" "bu_workload" {
  ami           = data.aws_ami.al2023.id
  instance_type = "t3.micro"

  network_interface {
   network_interface_id = aws_network_interface.bu_workload.id
   device_index         = 0
  }

  tags = {
   Name  = var.workload_name
   Owner = "business-unit"
  }
}
```

Plan and apply

31. From **c:\qatfadvaws\lab8\guided** run:

```
terraform plan
```

Confirm:

- Only Business Unit workload resources are being created
- No Landing Zone VPC or subnet resources are being created or changed

32. Apply the configuration:

```
terraform apply --auto-approve
```

Note: AWS Provider 6.x reports a deprecation warning for inline network interface attachment. The syntax remains functional and is retained in this lab for clarity and simplicity. Production implementations should consider aws_network_interface_attachment.

Verify Landing Zone consumption

In the AWS Console, confirm:

- The EC2 instance exists
- The workload network interface exists
- The workload security group exists
- The workload network interface is attached to the DEV Landing Zone subnet
- The Landing Zone VPC and subnet were not modified or imported

You have deployed a new workload correctly by referencing Landing Zone resources using data sources and keeping workload ownership separate from platform ownership.

Phase 5 - Migrate Imported Legacy Resources into the Landing Zone Model

In this phase you reconcile the Business Unit's imported legacy networking with the centrally managed DEV Landing Zone. Legacy bespoke networking must be retired, and workloads must consume the Landing Zone instead.

What you are about to do

- What: Migrate BU-owned resources away from bespoke networking and towards the Landing Zone network boundary.
- Why: Centralisation only works if legacy patterns are removed, not just prevented going forward.
- Success looks like: the legacy VPC, subnet, security group and ENI are removed from AWS only after a Landing Zone-aligned replacement exists.

Step 1 - Confirm legacy resources are still present and BU-owned

33. From **c:\qatfadvaws\lab8\guided,** run:

```
terraform state list
```

Expected outcome: you can see your imported legacy resources in state:

- aws_vpc.legacy
- aws_subnet.legacy
- aws_security_group.legacy
- aws_network_interface.legacy

This confirms the Business Unit currently owns the legacy resources in Terraform and therefore has the ability to retire them safely.

Step 2 - Plan the migration approach

The Landing Zone is immutable. You must not modify it. Because of that, the typical migration pattern is:

- Create a Landing Zone-aligned replacement, such as a new ENI in the Landing Zone subnet.
- Retire the legacy networking: old ENI, security group, subnet and VPC.

This is the realistic enterprise approach: you do not move the platform; you move off the legacy design.

Step 3 - Add a replacement ENI into the Landing Zone

34. In main.tf, add a replacement ENI alongside the existing resources:

```
resource "aws_network_interface" "legacy_replacement" {
 subnet_id      = data.aws_subnet.lz_app.id
 security_groups = [aws_security_group.bu_workload.id]

 tags = {
  Name  = "legacy-eni-replacement"
  Owner = "business-unit"
 }
}
```

This ENI is deliberately created in the Landing Zone subnet using data sources from Phase 4. You are not editing legacy resources yet. You are creating the safe destination first.

Step 4 - Plan and apply the replacement

35. Run:

```
terraform plan
```

Confirm the plan shows one resource to add and no Landing Zone resources to change.

36. Apply the creation of the replacement ENI:

```
terraform apply --auto-approve
```

37. In the AWS Console, verify:

- legacy-eni-replacement exists
- It is attached to the DEV Landing Zone subnet
- It uses the Business Unit workload security group

At this point you have proven a safe migration principle: create the compliant replacement first.

Step 5 - Retire the legacy networking

Now that a Landing Zone-aligned replacement exists, the legacy networking can be removed.

38. In main.tf, remove the legacy resources you imported earlier:

- aws_network_interface.legacy
- aws_security_group.legacy
- aws_subnet.legacy
- aws_vpc.legacy

39. Run: **terraform plan**

Expected outcome:

- 0 to add
- 0 to change
- 4 to destroy: legacy ENI, legacy security group, legacy subnet and legacy VPC

Terraform does not hide destructive change. It surfaces it clearly so that it can be governed.

Step 6 - Apply the retirement

40. Run: **terraform apply --parallelism=1 --auto-approve**
41. Confirm the destroys complete successfully.

If AWS prevents deletion because a dependency remains, review the plan and state carefully. In a real migration, dependency discovery is part of the reconciliation process.

Step 7 - Validate the end state

42. Run: **terraform state list**

Expected outcome:

- No legacy VPC, subnet, security group or ENI resources remain in state
- The Landing Zone replacement ENI remains
- The Phase 4 workload resources remain

43. In the AWS Console, verify:

- The legacy VPC, subnet, security group and ENI no longer exist
- The DEV Landing Zone VPC and subnet still exist and were not modified
- The workload EC2 instance, workload ENI and replacement ENI remain in the Landing Zone

Phase outcome: you recovered ownership of unmanaged resources, created new workload resources correctly in the Landing Zone, and retired legacy bespoke networking only after a compliant replacement existed.

Governance Discussion - Why Terraform Surfaces Risk Instead of Hiding It

This phase intentionally ended with destructive actions being clearly presented in the Terraform plan. That was not an accident. In centrally governed environments, the most dangerous changes are not the ones that Terraform blocks - they are the ones that happen silently.

What Terraform made visible

- Which resources would be destroyed
- That those resources were legacy, bespoke and no longer compliant
- That destruction would occur only after a Landing Zone-aligned replacement existed

This transparency enables informed decision-making rather than accidental change.

Why this matters in regulated environments

In real organisations, this plan output would typically trigger a formal approval workflow, change record, risk assessment or scheduled maintenance window. Terraform does not replace governance. It supports governance by providing deterministic information about impact and blast radius.

Platform vs workload responsibility revisited

At no point in this phase did the Business Unit:

- Modify Landing Zone networking
- Import Landing Zone resources
- Assume ownership of platform infrastructure

This aligns with the AWS platform operating model: platform teams own shared infrastructure, while workload teams consume, migrate and adapt within approved boundaries.

Why Terraform does not auto-fix legacy design

A common misconception is that Terraform should automatically migrate or refactor infrastructure into compliance. Terraform deliberately does not do this. Instead, it surfaces drift, highlights incompatibilities, makes destructive changes explicit and requires human approval for irreversible actions.

Key takeaway

Centralisation does not erase history. Terraform provides a mechanism to recover ownership, understand legacy decisions, introduce compliant patterns and retire outdated designs safely and deliberately. Progress happens through reconciliation, not denial.

This is why modern Terraform usage in enterprises focuses as much on governance, visibility and migration as it does on creation.

Lab Clean Up

This is that last lab of the course so destroying all lab resources to avoid unnecessary costs is recommended, especially if you are using you own AWS account.

Business Unit Backend S3 Bucket

```
cd c:\qatfadvaws\artifacts\bu-state-backend-bootstrap
terraform destroy --auto-approve
```

DEV environment resources

```
cd c:\pipeline-repos\env-roots\envs\dev
terraform init --upgrade
terraform destroy --auto-approve
```

TEST environment resources

```
cd c:\pipeline-repos\env-roots\envs\test
terraform init --upgrade
terraform destroy --auto-approve
```

PROD environment resources

```
cd c:\pipeline-repos\env-roots\envs\prod
terraform init --upgrade
terraform destroy --auto-approve
```

Business Unit Backend S3 Bucket

```
cd c:\qatfadvaws\artifacts\lz-state-backend-bootstrap
terraform destroy --auto-approve
```

Git Repositories

Delete the three repositories used at your own discretion.

Congratulations, you have completed this lab